

Manual de Desarrollo de Módulos

API REST Laravel + SQL Server — Sistema de Control de Asistencia, Red de Salud Trujillo

Para quién es esto. Para los cinco integrantes del equipo. Todos los módulos del sistema son, en el fondo, el mismo patrón repetido: leer, crear, actualizar y dar de baja filas de una o varias tablas, con reglas de negocio encima. Este manual describe ese patrón una sola vez para que no haya que redescubrirlo en cada módulo y para que el código de los cinco se parezca entre sí.

Lo que no es. No es el sistema escrito por adelantado. Son las indicaciones y convenciones; el contenido concreto de cada módulo lo pone quien lo desarrolle.

1. El cambio de mentalidad: ya no se crean tablas

En un proyecto Laravel normal escribes una migración, y esa migración *crea* la tabla. Acá no. La base de datos ya existe: la construyó `V001__esquema_base.sql` con sus 78 tablas, 13 esquemas, catálogos y vistas.

Eso cambia de dónde parte el trabajo:

Proyecto Laravel típico	Este proyecto
migración → modelo → controlador	modelo → request → controlador → ruta
la migración define la estructura	la estructura ya está; el modelo solo la <i>describe</i>
<code>php artisan migrate</code> crea tablas	<code>php artisan migrate</code> solo aplica ALTER nuevos

Las migraciones siguen existiendo, pero solo para **cambios** al esquema, y siempre como el archivo `V0XX_*.sql` + su migración-cáscara, como ya probaron. Para desarrollar un módulo sobre una tabla que ya existe, **no se toca nada de `database/`**.

El modelo pasa a ser el punto de partida. Y el modelo, en este proyecto, no inventa nada: es un espejo en PHP de algo que ya está decidido en SQL Server.

2. Anatomía de un módulo

Cada módulo son entre cuatro y seis archivos, siempre los mismos, siempre en el mismo orden:

1. `app/Models/<Esquema>/<Tabla>.php` ← describe la tabla
2. `app/Http/Requests/<Tabla>Request.php` ← valida lo que entra
3. `app/Http/Resources/<Tabla>Resource.php` ← da forma a lo que sale (opcional)
4. `app/Services/<Modulo>Service.php` ← reglas de negocio (solo si hacen falta)
5. `app/Http/Controllers/Api/<Tabla>Controller.php` ← orquesta
6. `routes/api.php` ← expone

La lógica de por qué ese orden: cada archivo depende del anterior. El request necesita saber qué columnas existe (modelo). El controlador necesita el request. La ruta necesita el controlador. Si lo haces al revés, terminas volviendo atrás a corregir.

Una forma útil de pensar cada capa:

- **Modelo** — *qué es esta fila y con qué se relaciona.*
- **Request** — *qué tiene derecho a entrar al sistema.*
- **Service** — *qué significa hacer esta operación en el negocio.*
- **Controller** — *quién hace qué y en qué orden, y qué HTTP se devuelve.*
- **Resource** — *qué ve el cliente.*

Si una clase empieza a hacer el trabajo de otra —por ejemplo, un controlador que valida a mano, o un modelo que decide reglas de negocio— es la señal de que algo está en el lugar equivocado.

3. Paso 1 — El Modelo

El modelo es una clase PHP que le dice a Eloquent cómo es una tabla. En este proyecto tiene que decir más de lo normal, porque el esquema no sigue las convenciones que Laravel asume por defecto.

3.1 Lo que Laravel supone, y por qué acá está mal

Laravel supone	En nuestro esquema
tabla en plural, minúsculas: <code>microreds</code>	<code>Organizacion.Microred</code>
PK llamada <code>id</code>	<code>MicroredId</code>
existen <code>created_at</code> y <code>updated_at</code>	no existen; la traza va por el esquema <code>Auditoria</code>
<code>BIT</code> se entrega como entero	el frontend espera <code>true/false</code>

Las cuatro se corrigen declarándolas. Si olvidas `$timestamps = false`, todo **INSERT** falla con *Invalid column name 'created_at'*. Es el primer error que va a encontrar cada uno.

3.2 Plantilla

```
<?php

namespace App\Models\Organizacion;

use Illuminate\Database\Eloquent\Model;

use Illuminate\Database\Eloquent\Relations\BelongsTo;

use Illuminate\Database\Eloquent\Relations\HasMany;

class EstablecimientoSalud extends Model

{

    // Eloquent parte por el punto y lo envuelve como [Organizacion].[EstablecimientoSalud]

    protected $table = 'Organizacion.EstablecimientoSalud';
```

```
protected $primaryKey = 'EessId';

public $incrementing = true;

protected $keyType = 'int';

// OBLIGATORIO en todos los modelos de este proyecto

public $timestamps = false;

// Solo estas columnas se aceptan en create() y update().

// La PK nunca va acá.

protected $fillable = [

    'MicroredId',

    'TipoEstablecimientoId',

    'EessCodigo',

    'EessNombre',

    'EessCodigoRenipres',

    'EessCategoria',

    'EessEstado',

];

protected $casts = [

    'MicroredId'      => 'integer',

    'TipoEstablecimientoId' => 'integer',

    'EessEstado'      => 'boolean', // BIT -> true/false

];
```

```

// --- Relaciones: se leen "este EESS pertenece a una Microred" ---

public function microred(): BelongsTo

{

    return $this->belongsTo(Microred::class, 'MicroredId', 'MicroredId');

}

public function tipoEstablecimiento(): BelongsTo

{

    return $this->belongsTo(TipoEstablecimiento::class, 'TipoEstablecimientoId',
'TipoEstablecimientoId');

}

public function vinculos(): HasMany

{

    return $this->hasMany(\App\Models\Personal\VinculoLaboral::class, 'EessId', 'EessId');

}

// --- Scopes: filtros reutilizables ---

public function scopeActivos($query)

{

    return $query->where('EessEstado', 1);

}

}

```

Como los nombres de columna no siguen la convención de Laravel, **hay que pasar siempre los dos últimos argumentos** de **belongsTo** y **hasMany** (clave foránea y clave local). Si los omite, Eloquent busca **microred_id** y no encuentra nada.

3.3 \$fillable: qué es y por qué importa

`$fillable` es la lista blanca de columnas que se pueden llenar desde datos del usuario. Sirve contra el *mass assignment*: si alguien manda en el JSON un campo que no debería controlar (`EessId`, o mañana un `EsAdministrador`), Eloquent lo ignora en vez de escribirlo.

Regla: la PK nunca va en `$fillable`, y las columnas que decide el sistema (fechas de proceso, usuario que aprobó, estados de flujo) tampoco — esas las asigna el controlador o el service explícitamente.

3.4 Modelos sobre vistas

El esquema ya trae vistas resueltas (`Organizacion.vw_Estructura`, `Asistencia.vw_FaltasJustificadas`, etc.). Se consumen como modelos de solo lectura:

```
class VistaEstructura extends Model

{

    protected $table = 'Organizacion.vw_Estructura';

    public $timestamps = false;

    protected $primaryKey = null;

    public $incrementing = false;

}
```

No las repliques con joins de Eloquent: ya están escritas y optimizadas en SQL.

3.5 Organización de carpetas

Un subdirectorio por esquema, espejando SQL Server:

app/Models/

|— Organizacion/ Microred.php, EstablecimientoSalud.php, ...

|— Personal/ Trabajador.php, VinculoLaboral.php, ...

|— Configuracion/ Turno.php, Horario.php, ...

└─ Seguridad/ Usuario.php, Rol.php, ...

└─ Asistencia/ Marcacion.php, AsistenciaDiaria.php, ...

Con 78 tablas, un solo directorio `app/Models/` plano se vuelve inmanejable a la tercera semana.

4. Paso 2 — El Form Request

4.1 Qué es exactamente

Un Form Request es una clase que se encarga de dos cosas antes de que el controlador se ejecute:

1. ¿Este usuario tiene permiso de hacer esto? — método `authorize()`
2. ¿Los datos que llegaron son válidos? — método `rules()`

Tú lo describiste como "donde se insertan los tipos de datos y las limitaciones de las tablas", y la intuición es correcta, pero conviene precisar el porqué: **las restricciones de SQL Server son la última defensa, no la primera**. Si dejas que sea la base la que rechace, el usuario recibe un error 500 con un mensaje de SQL Server en inglés hablando de `CK_EessCategoria`. Con un Request, recibe un 422 con un JSON legible que dice qué campo está mal y por qué.

El Request es el espejo en PHP de lo que el esquema ya garantiza en SQL.

Lo importante es que **se valida solo, sin que el controlador haga nada**: basta con ponerlo como parámetro del método. Si algo falla, Laravel devuelve 422 y el controlador ni siquiera llega a ejecutarse.

4.2 Cómo traducir el esquema a reglas

Esta tabla es la que van a usar todo el tiempo:

En el script SQL	Regla del Request
<code>NOT NULL</code>	<code>required</code> (en <code>store</code>)

En el script SQL	Regla del Request
NULL	nullable
NVARCHAR(100)	string,max:100
INT / BIGINT	integer
BIT	boolean
DATE	date
DATETIME2(0)	date_format:Y-m-d H:i:s
DECIMAL(5,2)	numeric
UNIQUE (Col)	Rule::unique(Modelo::class, 'Col')
FOREIGN KEY → Otra(Id)	Rule::exists(OtroModelo::class, 'Id')
CHECK (Col IN (...))	Rule::in([...])
CHECK (A >= B)	gte:campo_b o una regla propia

4.3 El detalle que les va a costar una tarde

Las reglas **unique** y **exists** en formato de texto **NO** aceptan nombres de tabla con **punto**. Laravel interpreta lo que está antes del punto como el nombre de una *conexión*, no como el esquema:

//  Laravel entiende: conexión "Organizacion", tabla "EstablecimientoSalud"

```
'EessCodigo' => 'unique:Organizacion.EstablecimientoSalud,EessCodigo',
```

// → Database connection [Organizacion] not configured.

La forma correcta es pasar **la clase del modelo**, no el nombre de la tabla. Laravel detecta que es un modelo y usa su **\$table**:

// 

```
use Illuminate\Validation\Rule;
```

```
'EessCodigo' => [Rule::unique(EstablecimientoSalud::class, 'EessCodigo')],
```

```
'MicroredId' => [Rule::exists(Microred::class, 'MicroredId')],
```

Esto aplica a **todas** las tablas del proyecto, porque todas viven en un esquema. Conviene que quede en el README del repo.

4.4 Plantilla

Un solo Request para **store** y **update**, distinguiendo por el método HTTP:

```
<?php

namespace App\Http\Requests;

use App\Models\Organizacion\EstablecimientoSalud;

use App\Models\Organizacion\Microred;

use App\Models\Organizacion\TipoEstablecimiento;

use Illuminate\Foundation\Http\FormRequest;

use Illuminate\Validation\Rule;

class EstablecimientoSaludRequest extends FormRequest
{
    public function authorize(): bool
    {
        // Mientras no exista el módulo de Seguridad, true.

        // Después: return $this->user()->puede('EESS_ESCRIBIR');

        return true;
    }

    public function rules(): array
```

```

{

$esCreacion = $this->isMethod('POST');

$id = $this->route('establecimiento')?->EessId;

return [

    'MicroredId' => [

        $esCreacion ? 'required' : 'sometimes',

        'integer',

        Rule::exists(Microred::class, 'MicroredId')->where('MicroredEstado', 1),

    ],

    'TipoEstablecimientoid' => [

        $esCreacion ? 'required' : 'sometimes',

        'integer',

        Rule::exists(TipoEstablecimiento::class, 'TipoEstablecimientoid'),

    ],

    'EessCodigo' => [

        $esCreacion ? 'required' : 'sometimes',

        'string', 'max:30',

        Rule::unique(EstablecimientoSalud::class, 'EessCodigo')->ignore($id, 'EessId'),

    ],

    'EessNombre' => [$esCreacion ? 'required' : 'sometimes', 'string', 'max:200],

    'EessCodigoRenipres' => ['nullable', 'string', 'max:20],

    'EessCategoria' => ['nullable', Rule::in(['I-1','I-2','I-3','I-4','II-1','II-2','III-1'])],

```

```

        'EessEstado' => ['sometimes', 'boolean'],

    ];

}

public function messages(): array
{
    return [

        'MicroredId.exists' => 'La microred indicada no existe o está inactiva.',

        'EessCodigo.unique' => 'Ya existe un establecimiento con ese código.',

    ];

}

}

```

Dos detalles que resuelven problemas reales:

- **->ignore(\$id, 'EessId')** en **unique**. Sin esto, al editar un EESS sin cambiarle el código, la validación se queja de que el código ya existe — chocando consigo mismo.
- **sometimes** en lugar de **required** para **PATCH**. **PATCH** significa *actualización parcial*: solo llegan los campos que cambian. Con **required** obligarías a reenviar el registro entero.

5. ¿Dónde va la lógica de negocio? (tu pregunta de los joins y procedimientos almacenados)

Tu intuición es correcta: va en Laravel, no en JavaScript. Y el motivo es de seguridad, no de comodidad: cualquiera puede llamar a `/api/...` con Thunder Client o Postman y saltarse completamente el frontend. Si una regla vive en React, esa regla es una sugerencia. Si vive en el API, es una regla.

Pero "en Laravel" admite cuatro lugares distintos, y elegir mal complica el mantenimiento. Este es el criterio:

5.1 Lectura simple con datos de tablas relacionadas → Eloquent con `with()`

El caso más común: mostrar asistencias con el nombre del trabajador y del establecimiento. No hace falta ningún join escrito a mano.

```
$asistencias = AsistenciaDiaria::query()
```

```
->with([

    'vinculoLaboral.trabajador:TrabajadorId,TrabajadorNombres,TrabajadorApellidoPaterno',

    'vinculoLaboral.establecimiento:EessId,EessNombre',

    'estadoAsistencia:EstadoAsistencialId,EstadoAsistenciaNombre',

])

->whereBetween('AsistenciaDiariaFecha', [$desde, $hasta])

->paginate(25);
```

`with()` hace *eager loading*: en vez de una consulta por fila (el problema N+1), ejecuta una consulta por relación. Es la opción por defecto y cubre el 80 % de las pantallas.

5.2 Reportes y consultas de varias tablas → una clase Query o el Query Builder

Cuando lo que necesitas no es "un modelo con sus relaciones" sino una tabla de resultados con agregaciones —el reporte mensual de tardanzas por establecimiento, por ejemplo— `with()` no sirve. Ahí se usa el Query Builder, pero **no dentro del controlador**: en una clase propia.

```
<?php
```

```
namespace App\Queries;
```

```
use Illuminate\Support\Facades\DB;
```

```

class ReporteTardanzasQuery
{
    public function porEstablecimiento(int $anio, int $mes)
    {
        return DB::table('Asistencia.AsistenciaDiaria as ad')

            ->join('Personal.VinculoLaboral as vl', 'vl.VinculoLaboralId', '=', 'ad.VinculoLaboralId')

            ->join('Organizacion.EstablecimientoSalud as e', 'e.EessId', '=', 'vl.EessId')

            ->join('Asistencia.EstadoAsistencia as ea', 'ea.EstadoAsistencialId', '=',
'ad.EstadoAsistencialId')

            ->whereYear('ad.AsistenciaDiariaFecha', $anio)

            ->whereMonth('ad.AsistenciaDiariaFecha', $mes)

            ->where('ea.EstadoAsistenciaEsDescontable', 1)

            ->groupBy('e.EessId', 'e.EessNombre')

            ->select(

                'e.EessId',

                'e.EessNombre',

                DB::raw('COUNT(*) AS TotalEventos'),

                DB::raw('SUM(ad.AsistenciaDiariaMinutosTardanza) AS MinutosTardanza')

            )

            ->orderByDesc('MinutosTardanza')

            ->get();
    }
}

```

```
}
```

El controlador queda en tres líneas y la consulta se puede probar, reusar y optimizar por separado.

Antes de escribir un join, revisa si ya existe una vista. El esquema trae `vw_Estructura`, `vw_FaltasJustificadas` y otras. Si la vista responde tu pregunta, úsala.

5.3 Operaciones que tocan varias tablas → un Service con transacción

Cuando una acción del negocio implica más de un `INSERT/UPDATE`, va en un Service. Ejemplo real: aprobar una justificación de falta cambia `JustificacionFalta` y `AsistenciaDiaria` — y si una de las dos falla, ninguna debe quedar aplicada.

```
<?php

namespace App\Services;

use App\Models\Asistencia\AsistenciaDiaria;

use App\Models\Asistencia\EstadoAsistencia;

use App\Models\Asistencia\JustificacionFalta;

use Illuminate\Support\Facades\DB;

class JustificacionService
{
    public function aprobar(JustificacionFalta $justificacion, int $usuarioid, ?string $observacion):
    JustificacionFalta
    {
        return DB::transaction(function () use ($justificacion, $usuarioid, $observacion) {

            if ($justificacion->JustificacionFaltaEstado !== 'PENDIENTE') {

                throw new \DomainException('Solo se pueden aprobar justificaciones pendientes.');
```

```

    }

    $justificacion->update([

        'JustificacionFaltaEstado' => 'APROBADO',

        'UsuarioResolucionId'    => $usuarioid,

        'JustificacionFaltaFechaResolucion' => now(),

        'JustificacionFaltaObservacion'    => $observacion,

    ]);

    $faltaJustificada = EstadoAsistencia::where('EstadoAsistenciaCodigo',
    'FALTA_JUST')->firstOrFail();

    AsistenciaDiaria::where('VinculoLaboralId', $justificacion->VinculoLaboralId)

    ->whereBetween('AsistenciaDiariaFecha', [

        $justificacion->JustificacionFaltaFechaInicio,

        $justificacion->JustificacionFaltaFechaFin,

    ])

    ->update([

        'EstadoAsistenciald' => $faltaJustificada->EstadoAsistenciald,

        'JustificacionFaltald' => $justificacion->JustificacionFaltald,

    ]);

    return $justificacion->fresh();

});

}

}

```

`DB::transaction()` es la pieza clave: o se aplican todos los cambios, o ninguno. Sin ella, una falla a mitad de camino deja la base inconsistente — una justificación aprobada cuyas asistencias siguen marcadas como falta injustificada.

5.4 Procesos masivos por lotes → ahí sí, considerar procedimiento almacenado

Esta es la parte de tu pregunta sobre procedimientos almacenados, y la respuesta honesta es: **rara vez, pero existe un caso legítimo en este sistema.**

El criterio no es "¿lleva joins?" sino "**¿tiene sentido mover estos datos a PHP?**".

- Si el resultado son 25 filas para una pantalla → Eloquent o Query Builder. Traerlas a PHP es barato.
- Si el proceso lee 40 000 marcaciones, las cruza con turnos programados y tramos de tolerancia, y escribe 15 000 filas de `AsistenciaDiaria` → traer todo a PHP, calcular fila por fila y devolverlo es lento y consume memoria. Eso es trabajo *set-based*, y SQL Server lo hace mejor.

En este sistema los candidatos reales son dos, y ambos llegan tarde en el proyecto:

- El **motor de consolidación diaria**: marcaciones × turno programado × tramo de tolerancia → `AsistenciaDiaria`.
- El **consolidado mensual**: `AsistenciaDiaria` → `ConsolidadoAsistencia` + `DetalleConsolidado`.

Si deciden usar un SP, se invoca desde un Service, nunca desde el controlador, y **el SP se versiona como `V0XX__sp_*.sql`** igual que cualquier otro cambio de esquema:

```
$filas = DB::select('EXEC Asistencia.sp_ConsolidarDia @Fecha = ?, @EessId = ?', [$fecha, $eessId]);
```

Mi recomendación para el proyecto: **escriban todo en PHP primero**. Es más fácil de probar, de versionar y de entender para los cinco. Muevan a SP solo si miden que un proceso concreto es demasiado lento. Optimizar antes de medir es la forma más común de complicar un proyecto sin ganar nada.

5.5 Resumen

Lo que necesitas	Dónde va
Listar con datos relacionados	Modelo + <code>with()</code> en el controlador

Lo que necesitas	Dónde va
Reporte con agregaciones o joins	Clase en <code>app/Queries/</code>
Consulta ya resuelta en el esquema	Vista SQL como modelo de solo lectura
Acción que toca 2+ tablas	Service + <code>DB::transaction()</code>
Regla de negocio sobre los datos	Service
Validación de forma y existencia	Form Request
Proceso batch de miles de filas	Job/Command; SP solo si se midió que hace falta
Cualquier regla de negocio	Nunca en React

6. Paso 3 — El Controlador

El controlador **orquesta**: recibe, delega, responde. No valida (eso es el Request), no calcula (eso es el Service), no arma consultas complejas (eso es la Query).

```
php artisan make:controller Api/EstablecimientoSaludController --api
```

`--api` genera los cinco métodos REST sin `create` ni `edit` (que devuelven formularios HTML, inútiles en un API).

```
<?php
```

```
namespace App\Http\Controllers\Api;
```

```
use App\Http\Controllers\Controller;
```

```
use App\Http\Requests\EstablecimientoSaludRequest;
```

```
use App\Http\Resources\EstablecimientoSaludResource;
```

```
use App\Models\Organizacion\EstablecimientoSalud;
```

```
use Illuminate\Http\JsonResponse;
```

```

use Illuminate\Http\Request;

class EstablecimientoSaludController extends Controller
{
    // GET /api/establecimientos

    public function index(Request $request)
    {
        $eess = EstablecimientoSalud::query()

        ->with(['microred:MicroredId,MicroredNombre',
'tipoEstablecimiento:TipoEstablecimientoId,TipoEstablecimientoNombre'])

        ->when($request->filled('microred_id'), fn ($q) =>

            $q->where('MicroredId', $request->integer('microred_id')))

        ->when($request->filled('buscar'), fn ($q) =>

            $q->where(fn ($s) => $s->where('EessNombre', 'like',
'%' . $request->string('buscar') . '%')

                ->orWhere('EessCodigo', 'like', '%' . $request->string('buscar') . '%')))

        ->when($request->filled('estado'), fn ($q) =>

            $q->where('EessEstado', $request->boolean('estado')))

        ->orderBy('EessNombre')

        ->paginate($request->integer('por_pagina', 15));

        return EstablecimientoSaludResource::collection($eess);
    }

    // POST /api/establecimientos

    public function store(EstablecimientoSaludRequest $request): JsonResponse

```

```

{
    $eess = EstablecimientoSalud::create($request->validated());

    return (new EstablecimientoSaludResource($eess->load('microred', 'tipoEstablecimiento')))

        ->response()

        ->setStatusCode(201);
}

// GET /api/establecimientos/{establecimiento}

public function show(EstablecimientoSalud $establecimiento)
{
    return new EstablecimientoSaludResource(

        $establecimiento->load('microred', 'tipoEstablecimiento')

    );
}

// PUT|PATCH /api/establecimientos/{establecimiento}

public function update(EstablecimientoSaludRequest $request, EstablecimientoSalud
$establecimiento)
{
    $establecimiento->update($request->validated());

    return new EstablecimientoSaludResource($establecimiento->fresh()->load('microred'));
}

// DELETE /api/establecimientos/{establecimiento}

public function destroy(EstablecimientoSalud $establecimiento): JsonResponse

```

```

{

    // BAJA LÓGICA, no DELETE físico: hay FK que dependen de esta fila.

    $establecimiento->update(['EessEstado' => 0]);

    return response()->json(['mensaje' => 'Establecimiento desactivado.'], 200);

}

}

```

6.1 Tres decisiones que conviene que sean iguales en los cinco módulos

destroy desactiva, no borra. Casi toda tabla de este esquema tiene una columna `<Tabla>Estado BIT`. Un **DELETE** físico va a chocar contra las FK —no se puede borrar un EESS que tiene vínculos laborales— y además destruiría historia que el Reglamento obliga a conservar. La única excepción son las tablas puente (`UsuarioRol`, `RolPermiso`, `HorarioDetalle`), donde borrar la fila sí es la operación correcta.

Route model binding con clave personalizada. Al escribir `EstablecimientoSalud $establecimiento` en la firma, Laravel busca solo la fila por su PK y devuelve 404 si no existe, sin que escribas `findOrFail`. Funciona porque el modelo declara `$primaryKey`.

Filtros con `when()`. Evita las cadenas de `if` dentro del controlador y deja la consulta legible.

7. Paso 4 — El API Resource

El Resource decide **qué se ve** en el JSON. Es opcional, pero sin él devuelves la fila cruda con los nombres de columna del esquema, y cada cambio interno rompe el frontend.

```
php artisan make:resource EstablecimientoSaludResource
```

```
public function toArray(Request $request): array
```

```

{

    return [

```

```

'id'      => $this->EessId,

'codigo'  => $this->EessCodigo,

'nombre'  => $this->EessNombre,

'categoria' => $this->EessCategoria,

'renipres' => $this->EessCodigoRenipres,

'activo'  => (bool) $this->EessEstado,

'microred' => $this->whenLoaded('microred', fn () => [

    'id'    => $this->microred->MicroredId,

    'nombre' => $this->microred->MicroredNombre,

]),

'tipo' => $this->whenLoaded('tipoEstablecimiento', fn () => [

    'id'    => $this->tipoEstablecimiento->TipoEstablecimientoId,

    'nombre' => $this->tipoEstablecimiento->TipoEstablecimientoNombre,

]),

];

}

```

Gana tres cosas: nombres cómodos para React (**nombre** en vez de **EessNombre**), **BIT** convertido a booleano de verdad, y **whenLoaded()** que evita el problema N+1 — la relación solo aparece si el controlador la cargó.

Decidan esto ahora, no después. Si tres personas devuelven **EessNombre** y dos devuelven **nombre**, el frontend tendrá que manejar los dos formatos. Sugerencia: campos en **snake_case** y en español, sin el prefijo de la tabla.

8. Paso 5 — La ruta API

En Laravel 11+ el archivo `routes/api.php` no viene por defecto. Una sola vez en el proyecto:

```
php artisan install:api
```

Luego, por cada módulo, una línea:

```
<?php
```

```
use App\Http\Controllers\Api\EstablecimientoSaludController;
```

```
use App\Http\Controllers\Api\MicroredController;
```

```
use Illuminate\Support\Facades\Route;
```

```
Route::apiResource('microredes', MicroredController::class);
```

```
Route::apiResource('establecimientos', EstablecimientoSaludController::class);
```

`apiResource` genera las cinco rutas de golpe:

Método	URL	Acción
GET	<code>/api/establecimientos</code>	index
POST	<code>/api/establecimientos</code>	store
GET	<code>/api/establecimientos/{id}</code>	show
PUT / PATCH	<code>/api/establecimientos/{id}</code>	update
DELETE	<code>/api/establecimientos/{id}</code>	destroy

Si un módulo necesita acciones que no son CRUD, se declaran **antes** del `apiResource` (si no, `/aprobar` se confunde con un `{id}`):

```
Route::post('justificaciones/{justificacion}/aprobar', [JustificacionController::class, 'aprobar']);
```

```
Route::post('justificaciones/{justificacion}/rechazar', [JustificacionController::class, 'rechazar']);
```

```
Route::apiResource('justificaciones', JustificacionController::class);
```

Comprobar siempre con:

```
php artisan route:list --path=establecimientos
```

Convención de URLs del equipo: plural, minúsculas, sin tildes ni ñ, en español.
establecimientos, trabajadores, vinculos, papeletas, justificaciones,
programaciones.

9. Paso 6 — Pruebas con Thunder Client

Un módulo no está listo cuando compila. Está listo cuando las pruebas pasan y **los datos están de verdad en SQL Server**.

9.1 Preparar el entorno

En Thunder Client → *Env* → nuevo entorno **Local**:

Variable	Valor
baseUrl	http://127.0.0.1:8000/api
eessId	(vacía, se llena sola)
token	(vacía, hasta que exista Seguridad)

Y en cada petición, pestaña *Headers*:

Accept: application/json

Content-Type: application/json

Accept: application/json no es opcional. Sin ese header, cuando una validación falla Laravel devuelve una redirección HTML en vez del JSON 422, y parece que el endpoint está roto cuando en realidad funciona bien.

9.2 La secuencia mínima por módulo

Trece peticiones, en este orden, dentro de una colección con el nombre del módulo:

#	Método	URL	Qué comprueba	Espera
01	GET	{{baseUrl}}/establecimientos	lista y paginación	200
02	GET	{{baseUrl}}/establecimientos?microred_id=1	filtro por FK	200
03	GET	{{baseUrl}}/establecimientos?buscar=esperanza	búsqueda	200
04	POST	{{baseUrl}}/establecimientos	creación válida	201
05	POST	{{baseUrl}}/establecimientos	repetir el mismo código	422
06	POST	{{baseUrl}}/establecimientos	sin campos obligatorios	422
07	POST	{{baseUrl}}/establecimientos	MicroredId inexistente	422
08	POST	{{baseUrl}}/establecimientos	EessCategoría fuera del CHECK	422

#	Método	URL	Qué comprueba	Espera
09	POST	{{baseUrl}}/establecimientos	nombre de 300 caracteres	422
10	GET	{{baseUrl}}/establecimientos/{{eessId}}	detalle	200
11	GET	{{baseUrl}}/establecimientos/999999	inexistente	404
12	PATCH	{{baseUrl}}/establecimientos/{{eessId}}	actualización parcial	200
13	DELETE	{{baseUrl}}/establecimientos/{{eessId}}	baja lógica	200

Las pruebas 05 a 09 son las que importan de verdad. Que el camino feliz funcione se ve en dos minutos; lo que distingue un módulo terminado de uno a medias es que **rechace bien lo que debe rechazar**. Cada una de esas cinco corresponde a una restricción que existe en el script SQL: `UNIQUE`, `NOT NULL`, `FOREIGN KEY`, `CHECK`, `NVARCHAR(n)`.

9.3 Encadenar peticiones

En la prueba 04, pestaña *Tests*, para guardar el id creado y reusarlo:

Tipo	Campo	Acción	Valor
ResponseCode		equal	201
SetEnvVar	<code>json.data.id</code>	setTo	<code>eessId</code>

Así las pruebas 10 a 13 trabajan sobre el registro que acaba de crear la 04, y toda la colección corre de un tirón con *Run All*.

9.4 La comprobación que no se puede saltar

Después de `Run All`, abrir **SSMS** y mirar la tabla:

```
SELECT TOP 20 * FROM Organizacion.EstablecimientoSalud ORDER BY EessId DESC;
```

Thunder Client puede devolver 201 y aun así haber guardado algo distinto de lo esperado: un `BIT` en 0 cuando debía ser 1, un campo que el `$fillable` descartó en silencio, una fecha con el año equivocado. El 201 dice que la petición se procesó; solo la tabla dice qué se guardó.

10. Convenciones del equipo

Que las escriban una vez en el README y que los cinco las sigan.

10.1 Códigos HTTP

Código	Cuándo
200	GET, PUT/PATCH y DELETE correctos
201	POST que creó un recurso
204	operación correcta sin cuerpo de respuesta
401	no autenticado (falta o venció el token)
403	autenticado pero sin permiso
404	el recurso no existe
409	conflicto de estado (aprobar algo ya aprobado)
422	validación fallida — lo devuelve Laravel solo
500	error no controlado (nunca debería salir en pruebas)

10.2 Forma de la respuesta

Éxito en listado (lo genera `Resource::collection(...)->paginate()`):

```
{  
  
  "data": [ { "id": 1, "nombre": "C.S. La Esperanza" } ],  
  
  "links": { "first": "...", "last": "...", "prev": null, "next": "..." },  
  
  "meta": { "current_page": 1, "per_page": 15, "total": 42 }  
  
}
```

Error de validación (lo genera Laravel):

```
{  
  
  "message": "Los datos proporcionados no son válidos.",  
  
  "errors": { "EessCodigo": ["Ya existe un establecimiento con ese código."] }  
  
}
```

Error de negocio lanzado desde un Service:

```
{ "message": "Solo se pueden aprobar justificaciones pendientes." }
```

10.3 Ramas y commits

Una rama por módulo: `feature/m03-personal`, `feature/m04-seguridad`. Commits en formato convencional: `feat(personal): CRUD de Trabajador con validación de DNI`. PR a `develop`, revisado por alguien que no sea el autor.

11. Checklist: ¿el módulo está listo para pasar a React?

No se empieza la vista hasta que las diez casillas estén marcadas.

- ☐ Modelo con `$table` con esquema, `$primaryKey`, `$timestamps = false`, `$fillable` y `$casts`
 - ☐ Relaciones declaradas con FK y clave local explícitas, verificadas con `php artisan tinker`
 - ☐ Request que refleja todas las restricciones del script: `NOT NULL`, `UNIQUE`, `CHECK`, `FOREIGN KEY`, longitudes
 - ☐ `Rule::unique` / `Rule::exists` usando **clase de modelo**, no nombre de tabla
 - ☐ Controlador sin lógica de negocio; joins en `app/Queries/`, transacciones en `app/Services/`
 - ☐ `destroy` hace baja lógica, no `DELETE`
 - ☐ Resource con nombres de campo acordados con el equipo
 - ☐ `php artisan route:list` muestra las rutas esperadas
 - ☐ Colección de Thunder Client con las 13 peticiones, y `Run All` en verde
 - ☐ **Verificado en SSMS que los datos quedaron correctos en la tabla**
-

12. Errores frecuentes

Mensaje	Causa	Solución
Invalid column name 'created_at'	falta <code>public \$timestamps = false</code>	agregarlo al modelo
Invalid column name 'updated_at'	lo mismo, al actualizar	ídem
Database connection [Organizacion] not configured	<code>unique:/exists:</code> con nombre de tabla con punto	usar <code>Rule::unique(Modelo::class, 'Col')</code>
Add [Campo] to fillable property	la columna no está en <code>\$fillable</code>	agregarla
Class "App\Http\Controllers\Api\XController" not found	el namespace no coincide con la carpeta	mover el archivo a <code>Api/</code> o corregir el namespace, y <code>composer dump-autoload</code>
Invalid object name 'Microred'	falta el esquema en <code>\$table</code>	<code>'Organizacion.Microred'</code>

Mensaje	Causa	Solución
Devuelve HTML en vez de JSON al fallar validación	falta el header <code>Accept: application/json</code>	agregarlo en Thunder Client
Relación siempre vacía	faltan los argumentos de FK y clave local	<code>belongsTo(Modelo::clas s, 'FkId', 'PkId')</code>
<code>The INSERT statement conflicted with the FOREIGN KEY constraint</code>	se está insertando un id que no existe	agregar <code>Rule::exists</code> al Request; revisar el orden de módulos
La lista tarda muchísimo	problema N+1	<code>with()</code> en el controlador
404 en una ruta que existe	la ruta específica está después del <code>apiResource</code>	moverla antes

13. Flujo de trabajo acordado

1. Elegir el siguiente módulo según la Hoja de Ruta de Dependencias
2. Confirmar que las tablas de las que depende YA tienen datos
3. Modelo → Request → (Service/Query si hace falta) → Controller → Resource → Ruta
4. Colección Thunder Client, Run All en verde
5. Verificar en SSMS
6. PR a develop, revisado por otro integrante
7. Recién entonces: la vista en React de ese módulo
8. Volver al paso 1

Un módulo a la vez, backend y frontend completos, antes de empezar el siguiente. Es más lento al principio y mucho más rápido a partir del tercer módulo, porque el patrón ya está resuelto y cada uno es copiar y adaptar.

Documento de trabajo del equipo. Actualizar cuando una convención cambie: un manual desactualizado es peor que no tenerlo.